Designing for High Availability

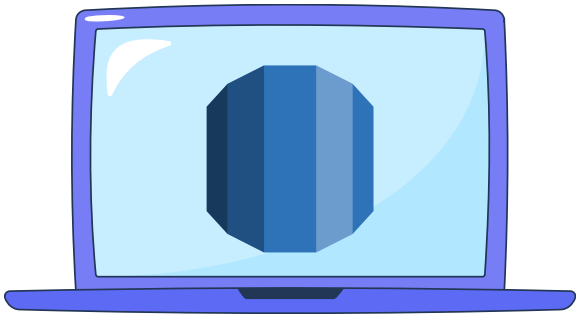
Explore how to design highly available applications on AWS using multi-AZ deployments, load balancing, and Auto Scaling. Understand active-active and active-passive patterns, AWS service features like RDS and Lambda, and strategies to handle failure scenarios. Gain practical knowledge on using AWS CLI and SDK to build fault-tolerant cloud solutions aligned with the AWS Developer Associate exam.

High availability (HA) refers to systems that remain operational and accessible, even when parts of the infrastructure fail. In AWS, this principle is central to how services are designed and consumed. For developers, HA means applications continue running smoothly without interruption, even in the face of hardware failures, network issues, or unexpected demand spikes.

A common misconception is that simply deploying an application on AWS guarantees high availability. AWS provides the building blocks, but developers must architect for resilience. For example:

- Running a single EC2 instance in one Availability Zone (AZ) might work, but it creates a single point of failure.
- By contrast, deploying across multiple AZs or even multiple Regions, combined with services like Elastic Load Balancing and Auto Scaling, allows applications to recover automatically if one component fails.

Ask



Create and connect your Amazon RDS instance

Global infrastructure

AWS is built on a global infrastructure comprising Regions and Availability Zones. A **region** is a physical location with multiple, isolated data centers called AZs. Each AZ is a fully isolated partition of the AWS infrastructure, but it is connected with low-latency networking.

Architecting across Availability Zones

One of the most critical practices when designing for HA is AZ diversification. Services like Amazon EC2 Auto Scaling, Amazon RDS Multi-AZ, and Elastic Load Balancing make it easier to distribute resources.

For example, deploying a web tier across EC2 instances in two AZs and using an Application Load Balancer (ALB) to distribute requests ensures that even if one AZ goes down, our application remains online. While serverless and abstracted from infrastructure, AWS Lambda also benefits from multi-AZ redundancy automatically handled by AWS. To further improve availability, developers can combine Lambda with services like SQS and EventBridge for decoupled, fault-tolerant workflows.

Let's review some examples to show how we can architect resources across AZs using both AWS CLI and SDK, and programmatically connect resources to load balancers:

Ask

- **Creating an Auto Scaling group with AWS CLI:** This example creates an Auto Scaling group named `my-asg` with a specified launch configuration, desired instance count, scaling limits, and associated subnets for load distribution.

main.sh Shell-20

```shell
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name my-asg \
  --launch-configuration-name my-launch-config \
  --min-size 2 --max-size 5 --desired-capacity 3 \
  --vpc-zone-identifier "subnet-abc123,subnet-def456"
```

Example using AWS CLI

- **Register targets in ALB with Python SDK:** This example registers two EC2 instances with a specified target group using the Elastic Load Balancing v2 (ELBv2) API. This distributes incoming application traffic across the two instances in the target group.

main.py boto3

```python
import boto3
client = boto3.client('elbv2')

response = client.register_targets(
    TargetGroupArn='arn:aws:elasticloadbalancing:region:123456789
    Targets=[
        {'Id': 'i-abc123'},
        {'Id': 'i-def456'}
```

Ask

```
        ]
    )
```

Example using Boto3 (Python)

- **RDS Multi-AZ deployment using AWS CLI:** This example creates a Multi-AZ Amazon RDS MySQL DB instance with 20 GB of storage, a 7-day backup retention period, and a specified admin username and password. The instance that will automatically fail over to a standby in case of failure.

main.sh C++

```
aws rds create-db-instance \
  --db-instance-identifier mydb \
  --db-instance-class db.t3.micro \
  --engine mysql \
  --master-username admin \
  --master-user-password secret99 \
  --allocated-storage 20 \
  --multi-az \
  --backup-retention-period 7
```

Example using CLI

- **Lambda + SQS high availability trigger with Python SDK:** This example creates an event source mapping that triggers a Lambda function from an Amazon SQS queue with a batch size of 5.

Ask

main.py C++

```
import boto3
```

```
lambda_client = boto3.client('lambda')
event_source_mapping = lambda_client.create_event_source_mapping(
    EventSourceArn='arn:aws:sqs:us-west-2:123456789012:my-queue',
    FunctionName='my-lambda-function',
    Enabled=True,
    BatchSize=5
)
```

Example using Boto3 (Python)

This configuration triggers a Lambda function from an SQS queue, providing high availability by decoupling services and enabling automatic retry handling. However, we still need to consider downstream services (like RDS or S3) to make the full stack highly available.

Multi-instance and multi-AZ deployment patterns

Let's take a closer look at deployment strategies that support high availability in technical terms relevant to AWS service configurations:

Active-active

This pattern runs multiple instances or services across multiple AZs or even Regions. All nodes actively process traffic at the same time. Developers typically use Amazon EC2 behind an Application Load Balancer (ALB), combined with Auto Scaling groups that span subnets in multiple AZs. Stateless services or microservices often benefit from this design.

The CLI command given below creates an Application Load Balancer in the specified subnets with an associated security group.

Ask

main.sh Shell-20

```
aws elbv2 create-load-balancer \
  --name my-alb \
  --subnets subnet-abc123 subnet-def456 \
  --security-groups sg-0123456789abcdef0
```

Create an ALB across two subnets with a security group for traffic control

Active-passive

In this pattern, a primary resource handles all requests while a secondary remains in standby, ready to take over during failure. Amazon RDS Multi-AZ deployments are a canonical example—only the primary serves traffic, but AWS manages synchronous replication to the standby.

The CLI command given below provisions a Multi-AZ Amazon RDS PostgreSQL instance with 20 GB of storage and a specified instance class.

main.sh Shell-20

```
aws rds create-db-instance \
  --db-instance-identifier prod-db \
  --multi-az \
  --engine postgres \
  --master-username admin \
  --master-user-password secretpass \
  --allocated-storage 20 \
  --db-instance-class db.t3.medium
```

Ask

Launch a high-availability PostgreSQL RDS instance with admin credentials and automatic failover.

Choosing between these strategies depends on our application's characteristics:

- Use **active-active** for stateless applications that scale horizontally (e.g., APIs, web servers).
- Use **active-passive** for stateful services (e.g., RDS) where consistency and managed failover are priorities.

This distinction is a common topic on the DVA-C02 exam and directly reflects how developers are expected to reason about service behavior and failover automation.

Multi-instance and multi-AZ deployment patterns

High availability capabilities by the AWS service

Ask

AWS also provides built-in high availability mechanisms to provide high availability. This is important for developers to understand what is managed by

AWS and what still requires thoughtful application design.

- **Amazon S3:** Amazon S3 is built for 99.999999999% durability and high availability across multiple AZs by default. No extra configuration is required. For example, a media streaming service storing thumbnails and user-uploaded files in S3 can continue serving requests even during an AZ failure, since S3 automatically replicates data across facilities.
- **Amazon DynamoDB:** Amazon DynamoDB offers multi-AZ replication and on-demand backup for availability and durability. Its architecture makes it well-suited for high-volume use cases like gaming leaderboards or IoT telemetry. For example, a mobile game storing user scores in DynamoDB continues to respond quickly during an AZ disruption thanks to its distributed nature.
- **AWS Lambda:** AWS Lambda functions run in multiple AZs without user intervention. However, developers must still design around potential failures in event sources or downstream services. For example, a serverless image processing pipeline that invokes Lambda from an S3 upload event remains highly available—but care must be taken to handle errors from dependencies like RDS or third-party APIs gracefully.

Failure scenarios and resilience engineering

AWS provides various fault-tolerant features that help developers create applications resilient to common failure modes, such as AZ outages, instance crashes, or traffic spikes. To build truly fault-tolerant systems, we must anticipate what could go wrong and design graceful failure mechanisms. The following subsections break down the key AWS services used to handle failure scenarios, each with a real-world example.

- **Elastic Load Balancing:** It detects instance failures and redirects traf example, if an EC2 instance in one Availability Zone becomes unresponsive due to a hardware failure or OS issue, the load balancer automatically stops

Ask

routing requests to that instance and instead forwards them to healthy instances in other AZs. This ensures the application remains available without manual intervention.

- **Auto Scaling groups:** Auto Scaling groups replace unhealthy instances automatically. In a real-world scenario, if a web application hosted on EC2 experiences memory leaks or CPU exhaustion, causing a crash, the Auto Scaling Group detects the failure using health checks, terminates the affected instance, and launches a new one with the same configuration. This minimizes downtime and reduces the need for manual recovery steps.
- **Route 53 health checks:** Route 53 health checks switch DNS records away from unhealthy endpoints. Imagine a scenario where our primary application endpoint is hosted in us-east-1 and suffers a network failure. Route 53 health checks will detect the unresponsive endpoint and reroute DNS traffic to a healthy secondary region or static backup page in another AZ or region. This supports seamless failover and ensures continuity of service. By integrating monitoring, scaling, and health checks, our application can detect issues early and initiate automated recovery procedures.

> **Tip:** To detect failure points early, use AWS CloudWatch for observability and AWS X-Ray for tracing. Also, consider a decoupling architecture using Amazon SQS or EventBridge to isolate components and further improve resilience.

Conclusion

We explored critical strategies and AWS services that enable high availability, including multi-AZ deployments, auto scaling, load balancing, active-active and active-passive patterns, and resilient configurations using the AWS CLI and SDK. We've seen how services like EC2, RDS, Lambda, S3, and DynamoDB beh the context of failure recovery, and how to architect systems to minimize downtime and disruption.

Ask

These practices are essential in real-world cloud application design and align closely with the AWS Certified Developer—Associate (DVA-C02) exam. The exam

Ask